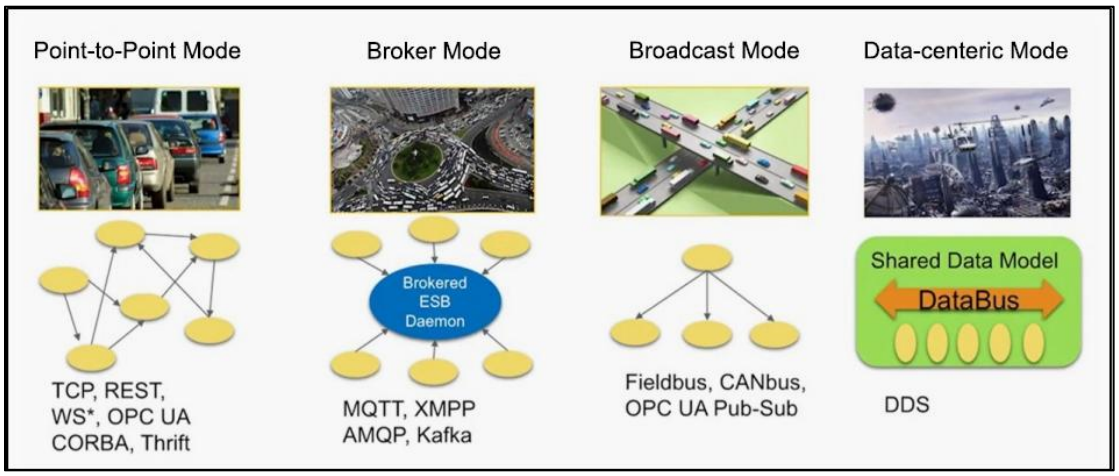
Lesson 14 ROS2 DDS Instruction

1. DDS Introduction

The full name of DDS is Data Distribution Service, which means data distribution service. It was published and maintained by the Object Management Organization (OMG) in 2004. It is a set of data publish-subscribe standards tailored for real-time systems. It was initially used by the US Navy to address compatibility issues with large software upgrades in complex naval network environments. It has since become a mandatory standard.

DDS emphasizes putting data at the center and can provide a rich set of quality of service policies to ensure real-time, efficient, and flexible data distribution. It can meet various requirements of distributed real-time communication applications.

In the previous courses, the topics, services, and actions learned are all implemented at the communication level through DDS. It can be considered as the neural network within the ROS system. The common communication models include the following four types:



1) In the point-to-point model, many clients connect to a single server, requiring the establishment of a connection each time communication occurs.

A the number of communication nodes increases, so does the number of connections. Additionally, each client needs to know the specific address of the server and the services it provides. Once the server's address changes, all clients are affected.

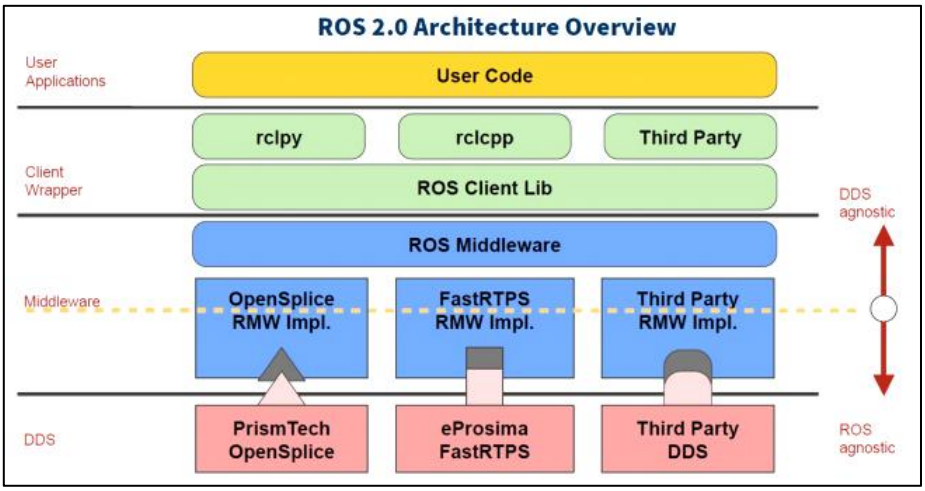
2) In the broker mode, optimization is applied to the point-to-point model. A centralized Broker handles all requests and further identifies the roles that can effectively respond to the service.This relieves clients from needing to concern themselves with the specific address of the server. However, there are evident issues with this approach. As the Broker serves as the core, its processing speed impacts the efficiency of all nodes. When the system scales to a certain extent, the Broker becomes the performance bottleneck of the entire system. Moreover, if the Broker encounters an exception, it may result in the entire system failing to operate normally. Previously, the ROS1 system utilized a similar architecture.

3) In the broadcast model, all nodes can broadcast on a channel and receive these messages. This model addresses the issue of server addresses, and eliminating the need for the communication parties to establish individual connections. However, the volume of messages on the broadcast channel is significant, requiring all nodes to process each message, even those unrelated to them.

4) The Data-Centric Model, similar to the broadcast model, allows all nodes to publish and subscribe to messages on the DataBus. However, its sophistication lies in incorporating multiple parallel channels of communication. Each node can focus solely focuses on the data itself rather than the endpoints involved in communication.

2. The Application of DDS in ROS

The position of DDS within the ROS 2 system is crucial, as all upper-level constructs are built upon it. In the architectural diagram of ROS 2, the blue and red sections represent DDS.



In the four major components of ROS, the addition of DDS significantly improves the comprehensive capability of the distributed communication system. Consequently, in the process of developing robots, we are relieved from the burden of grappling with communication intricacies, enabling us to allocate more time to application development in other domains.

3. Quality of Service (QoS)

The fundamental structure in DDS is the Domain, which binds various applications together for communication. Another important feature in DDS is Quality of Service - QoS.

QoS is a network transmission strategy where applications specify the required quality of network transmission behavior. QoS services fulfill these behavioral requirements, striving to meet customers' demands for communication quality. It can be regarded as a contract between data providers and receivers. The strategies are as follow:

◆ The DEADLINE policy indicates that communication data must be transmitted within a specified deadline for each communication instance.

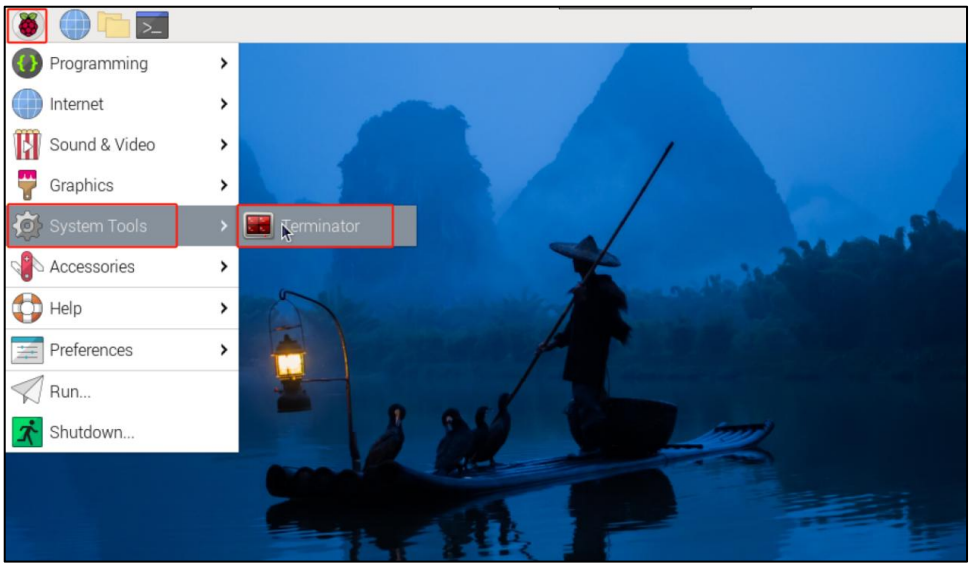
◆ The HISTORT policy indicates a cache capacity for historical data.

◆ The RELIABILITY policy indicates the mode of data communication. When it is configured as BEST_EFFORT, it operates in a best-effort transmission mode, ensuring data flow even under adverse network conditions, which may result in potential data loss. Configured as RELIABLE, it operates in a reliable mode, striving to maintain the integrity of the data during communication. We can choose the appropriate communication mode based on the functional scenarios of the application.

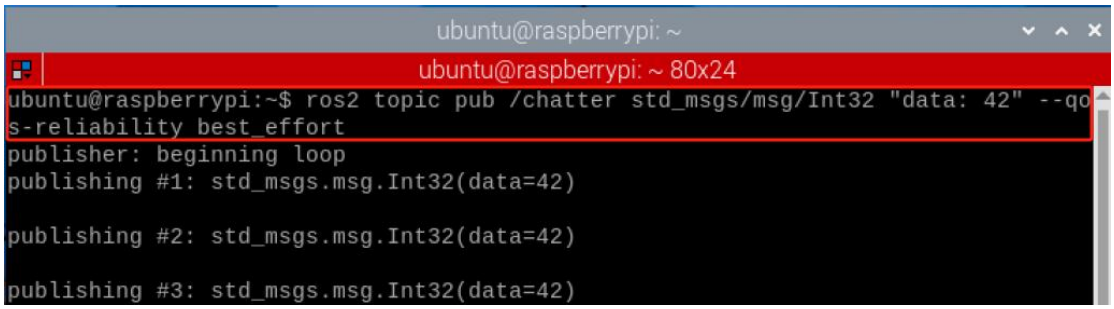
◆ The DURABILITY strategy allows configurations for nodes that are added later to ensure a certain amount of historical data is sent, enabling new nodes to quickly adapt to the system.

4. DDS Configuration in Command-line

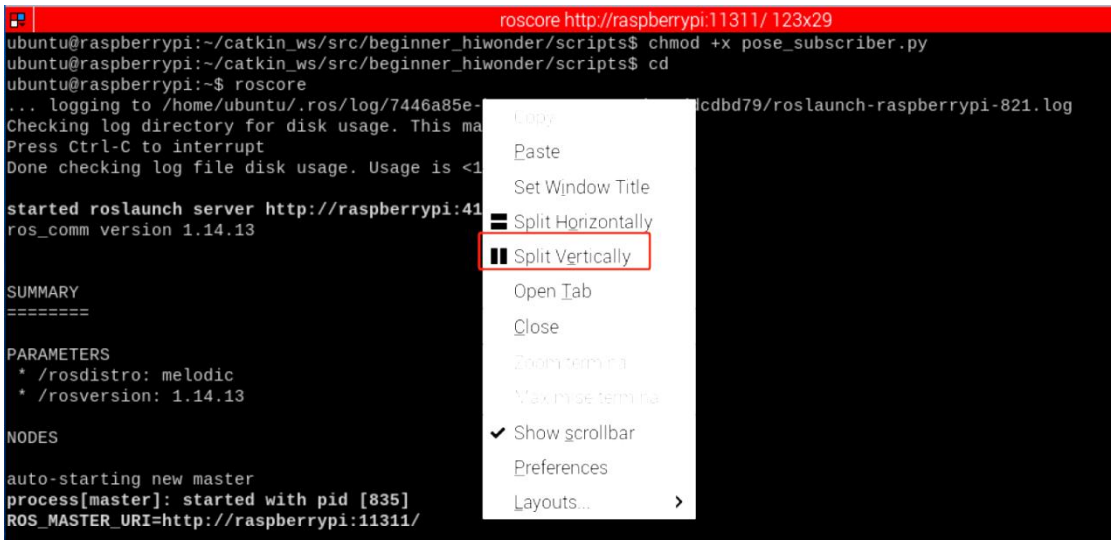
1) Click  in the upper-left corner to select “System Tools→Terminator” in sequence.



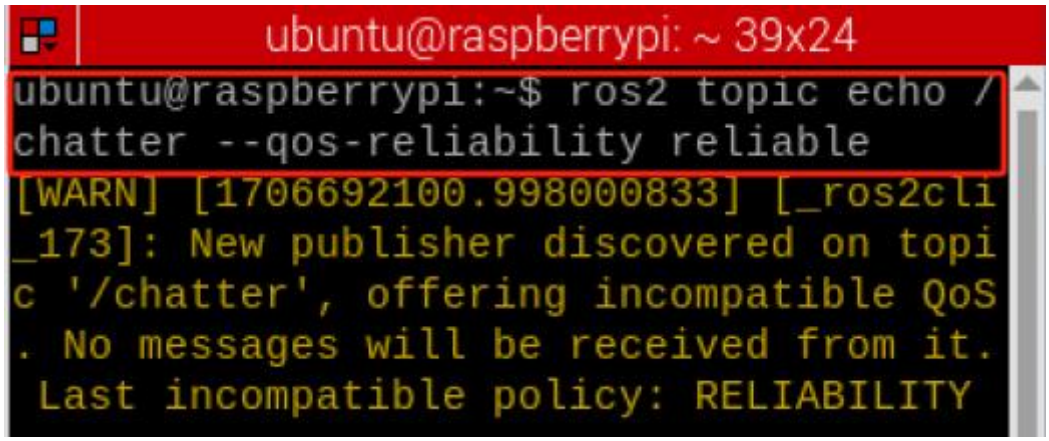
2) Enter the command “**ros2 topic pub /chatter std_msgs/msg/Int32 "data: 42" --qos-reliability best_effort**” to publish a topic named “/chatter” using the message type “**std_msgs/msg/Int32**”, sending an integer message with the data 42. Through adding the option of “--qos-reliability best_effort”, the publisher specifies the use of best effort reliability.



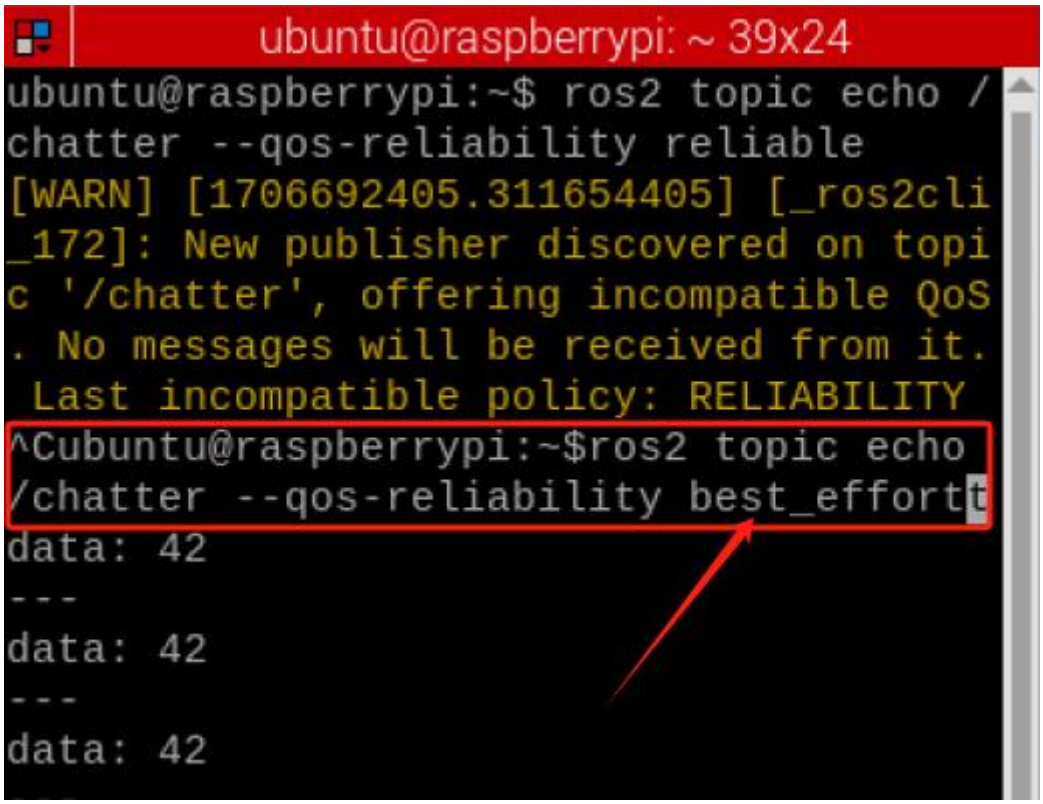
3) Right click on a blank area to select “**Split Vertically**” to create a new terminal window.



4) Enter the command “**ros2 topic echo /chatter --qos-reliability reliable**” to subscribe to a topic named "**/chatter**” and print the received messages. If the publisher uses the reliable QoS policy for publishing while the subscriber uses the best effort policy for subscribing, data communication cannot be achieve. Only when the publisher and receiver use the same QoS policy can the correct correct transmission of data be ensured.



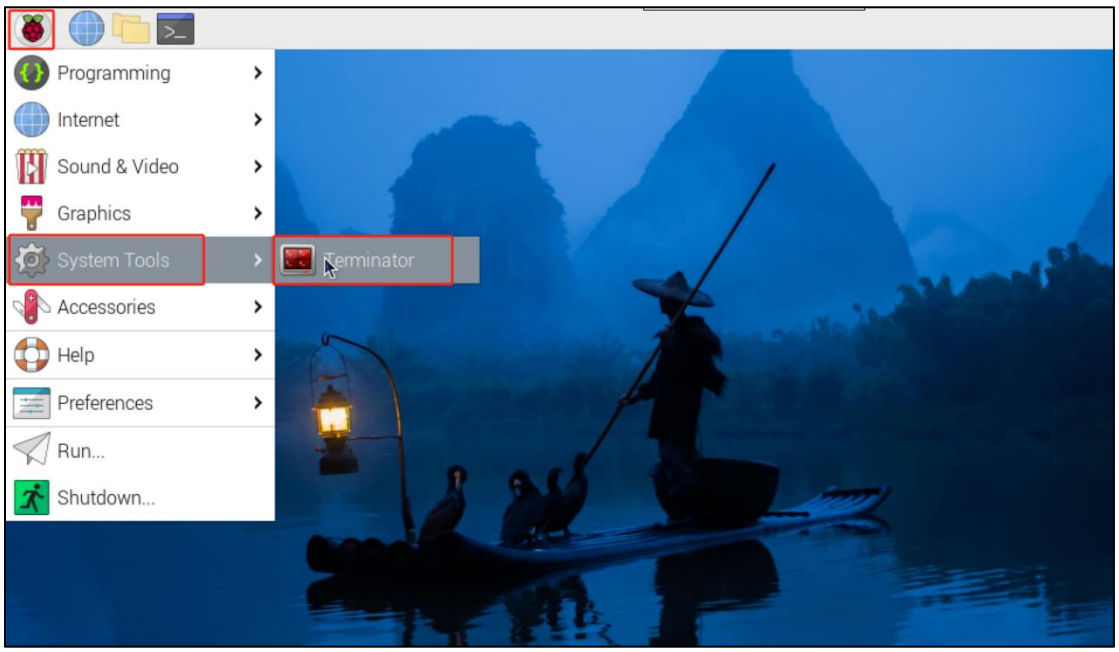
5) Enter the command "**ros2 topic echo /chatter --qos-reliability best_effort**" and subscribe to a topic named "**/chatter**" and print the received messages. By adding the "**--qos-reliability reliable**" option, modify it to the same "best_effort" in order to achieve data transmission



5. DDS Programming Example

5.1 Create Publisher

1) Click  in the upper-left corner to select “System Tools→Terminator” in sequence.



2) Enter command “**cd hiwonder_ws/src/**” to switch to the src folder within the hiwonder_ws workspace.

```
ubuntu@raspberrypi:~$ cd hiwonder_ws/src/
ubuntu@raspberrypi:~/hiwonder_ws/src$
```

3) Enter the command “**ros2 pkg create DDS_qos_demo --build-type ament_python --dependencies rclpy**” and press Enter to create a package named **DDS_qos_demo,** adding a dependency on rclpy.

```
ubuntu@raspberrypi:~/hiwonder_ws/src$ ros2 pkg create DDS_qos_demo --build-type
ament_python --dependencies rclpy
```

4) Enter the command “**cd DDS_qos_demo/DDS_qos_demo/**” to switch to the “DDS_qos_demo” package directory.

```
ubuntu@raspberrypi:~/hiwonder_ws/src$ cd DDS_qos_demo/DDS_qos_demo/
ubuntu@raspberrypi:~/hiwonder_ws/src/DDS_qos_demo/DDS_qos_demo$
```

5) Enter the command "**vim DDS_qos_pub.py**" to edit the program using VIM editor, copy and paste the program below. If you need to make modifications, you can press "i" to enter the insert mode. Once you've finished the modifications, you can press "Esc" and enter ":wq" to save and exit.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/DDS_qos_demo/DDS_qos_demo$ vim DDS_qos_pub.
py
```

```
import rclpy    # Import rclpy module
from rclpy.node import Node    # Import Node class


from std_msgs.msg import String    # Import the String message type.
from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy
#Import the QoSProfile, QoSReliabilityPolicy and QoSHistoryPolicy classes


class MinimalPublisher(Node):    # Define the MinimalPublisher class inhering from the Node class


    def __init__(self):
        super().__init__('minimal_publisher')    # Call the parent constructor to initialize the node.
        qos_profile = QoSProfile(    # Create the QoSProfile object
            reliability=QoSReliabilityPolicy.RELIABLE,    # Set reliability policy as RELIABLE
            history=QoSHistoryPolicy.KEEP_LAST,    # Set history policy as KEEP_LAST
```

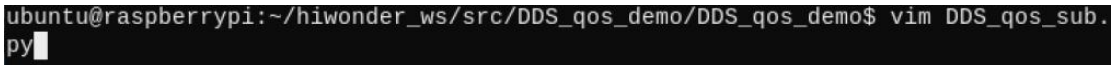
```
            depth=1    # Set the depth to 1
        )
        self.publisher_ = self.create_publisher(String, 'topic', qos_profile)    # Create publisher object
        timer_period = 0.5    # Set the the timer callback interval to 0.5 seconds
        self.timer = self.create_timer(timer_period, self.timer_callback)    # Create timer and set the callback function as timer_callback
        self.i = 0

    def timer_callback(self):
        msg = String()    # Create the object of the String message type
        msg.data = 'Hello World: %d' % self.i    # Set the message data
        self.publisher_.publish(msg)    # Publish message
        self.i += 1    # Increment the counter

def main(args=None):
    rclpy.init(args=args)    # Initialize ROS node

    minimal_publisher = MinimalPublisher()    # Create MinimalPublisher object

    rclpy.spin(minimal_publisher)    # Enter the main loop
```

```
    minimal_publisher.destroy_node()  # Destroy node
    rclpy.shutdown()  # Shut down ROS

if __name__ == '__main__':
    main()
```

6) Enter the command "**chmod +x DDS_qos_pub.py**" and press Enter to grant the executable permission to the saved "topic_pub.py".

5.2 Create Subscriber

1) Enter the command "**vim DDS_qos_sub.py**" to edit the program using VIM editor, copy and paste the program below. If you need to make modifications, you can press "i" to enter the insert mode. Once you've finished the modifications, you can press "Esc" and enter ":wq" to save and exit.

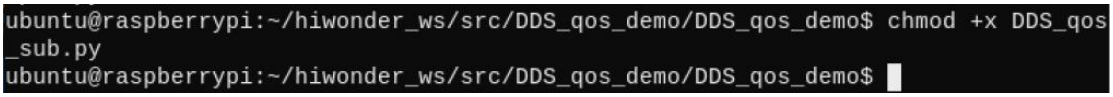



```
import rclpy  # Import rclpy
from rclpy.node import Node  # Import Node class

from std_msgs.msg import String  # Import String message type
```

```
from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy    # Import the QoSProfile, QoSReliabilityPolicy and QoSHistoryPolicy classes



class MinimalSubscriber(Node):    # Define the MinimalSubscriber class inhering from the Node class

    def __init__(self):
        super().__init__('minimal_subscriber')    # Call the parent constructor to initialize the node.

        qos_profile = QoSProfile(    # Create QoSProfile object
            reliability=QoSReliabilityPolicy.RELIABLE,    #Set the reliability policy as RELIABLE
            history=QoSHistoryPolicy.KEEP_LAST,    # Set the history policy as KEEP_LAST
            depth=1    # Set the depth to 1
        )
        self.subscription = self.create_subscription(String, 'topic', self.listener_callback, qos_profile)    # Set the subscriber object and set the callback function as listener_callback

    def listener_callback(self, msg):
        self.get_logger().info('I heard: "%s"' % msg.data)    # Print the
```

```
received message



def main(args=None):
    rclpy.init(args=args)  # Initialize ROS node

    minimal_subscriber = MinimalSubscriber()  # Create MinimalSubscriber
object

    rclpy.spin(minimal_subscriber)  # Enter the main loop

    minimal_subscriber.destroy_node()  # Destroy node
    rclpy.shutdown()  # Shut down ROS

if __name__ == '__main__':
    main()
```

1) Enter the command "**chmod +x DDS_qos_sub.py**" and press Enter to grant the executable permission to the saved file topic_sub.py.



6. setup.py File Settings

The setup.py file defines the metadata and build configuration for a ROS2 package, providing information such as package metadata, dependencies, build configuration, and installation logic. It helps developers correctly build, install, and use ROS2 packages. It is necessary to write the program entry points for topic_pub.py and topic_sub.py into the setup.py file.

1) Enter the command "**cd ..**" to navigate to the parent directory.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/DDS_qos_demo/DDS_qos_demo$ cd ..
ubuntu@raspberrypi:~/hiwonder_ws/src/DDS_qos_demo$
```

2) Enter the command "**vim setup.py**" and press Enter to open the setup.py file.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/DDS_qos_demo$ vim setup.py
```

3) Press "i" to enter the insert mode, and then enter the following code to the corresponding position.

```
'DDS_qos_pub = DDS_qos_demo.DDS_qos_pub:main',
'DDS_qos_sub = DDS_qos_demo.DDS_qos_sub:main'
```

```
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'DDS_qos_pub = DDS_qos_demo.DDS_qos_pub:main',
            'DDS_qos_sub = DDS_qos_demo.DDS_qos_sub:main'
        ],
    },
)
-- INSERT --                                              27,1          Bot
```

4) Enter "**:wq**" to save and exit the file.

```
        'console_scripts': [
            'DDS_qos_pub = DDS_qos_demo.DDS_qos_pub:main',
            'DDS_qos_sub = DDS_qos_demo.DDS_qos_sub:main'
        ],
    },
)
:wq
```

7. Compilation and Execution

1) After granting the executable permission, enter the command "**cd ~/hiwonder_ws/**" to switch to the directory of the workspace.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/hello_world_demo$ cd ~/hiwonder_ws/
ubuntu@raspberrypi:~/hiwonder_ws$
```

2) Enter the command "**colcon build**" and press Enter to compile the packages within the workspace.

```
ubuntu@raspberrypi:~/hiwonder_ws$ colcon build
```

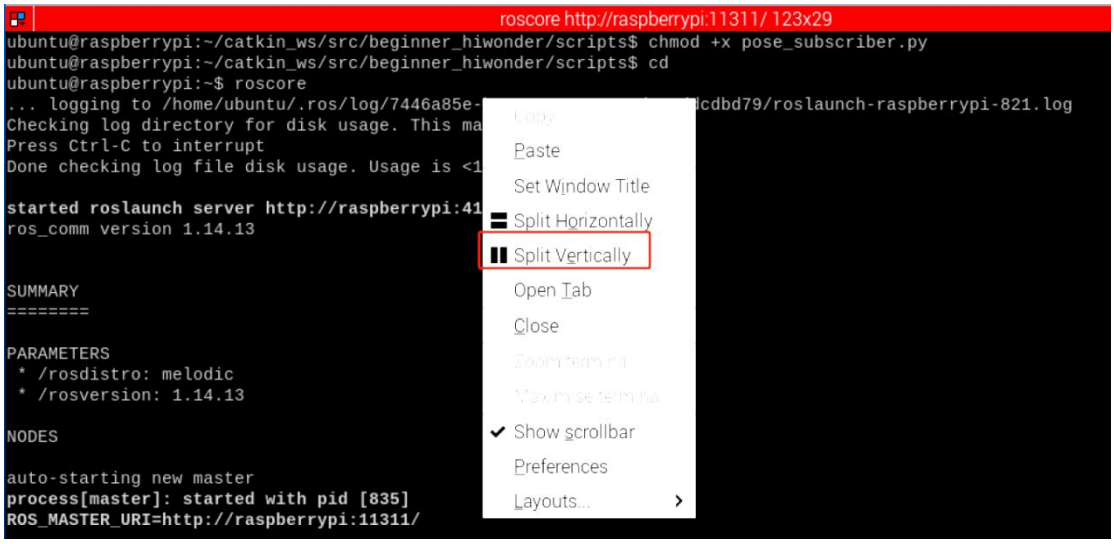
3) Enter the command "**source ./install/setup.bash**" and press Enter to make the environment variables take effect.

```
ubuntu@raspberrypi:~/hiwonder_ws$ source ./install/setup.bash
```

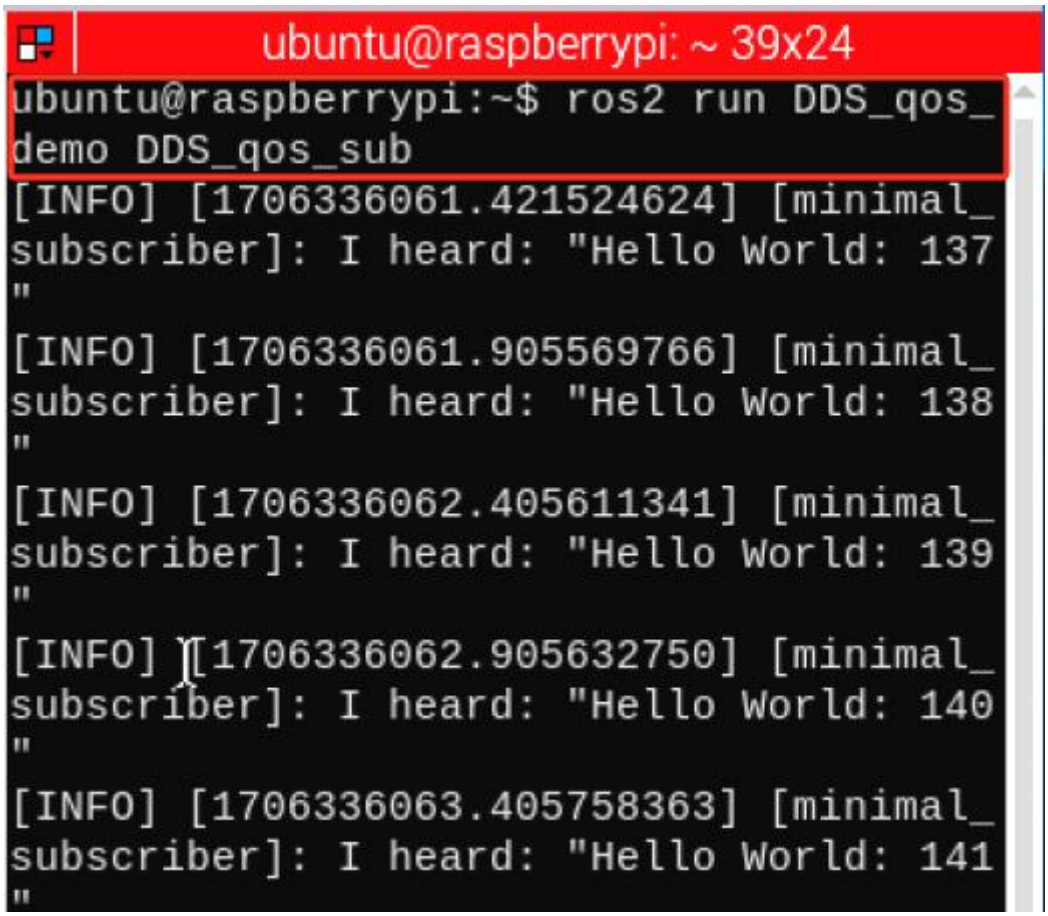
4) Enter the command "**ros2 run topic_demo topic_pub**" and press Enter to start topic_pub topic publishing node.

```
ubuntu@raspberrypi:~$ ros2 run DDS_qos_demo DDS_qos_pub
```

5) Right click on a blank space to select "**Split Vertically**" to create a new terminal window.



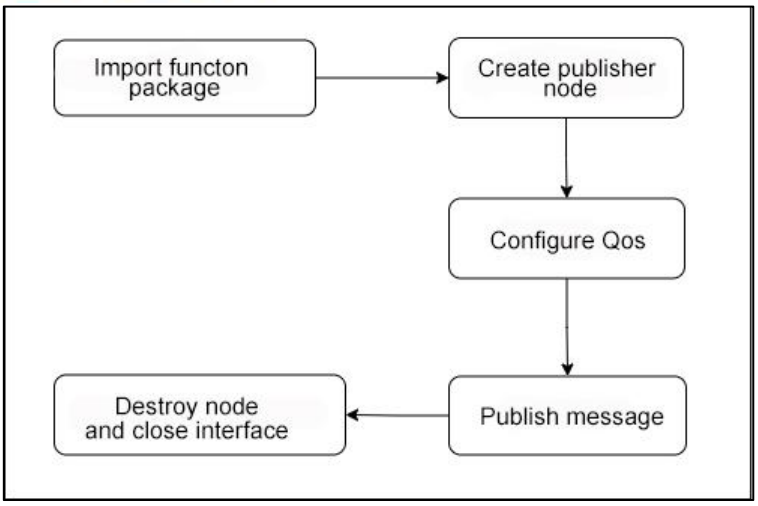
6) Enter the command “**ros2 run topic_demo topic_sub**” and press Enter to start the topic_sub topic publishing node.



8. Program Analysis

8.1 Publish Topic

According to the realization result, the logic progress for the program is shown as pictured:



A MinimalPublisher node class is created. In its constructor, a QosProfile object is instantiated to configuring publishing as reliable mode, ensuring the delivery of the lats or the first message, with a buffer depth set to 1. Then, a timer object with 0.5-sccond period is created. In the timer callback function, a String message is constructed, and its content is modified as the loop counter i increases. The message string is then published using the publisher_ object.

The main function first initializes the ROS node environment, then creates an instance of the node, and enters the main loop to drive the execution of the timer task. The timer task allows the published content to be sent in a loop. Finally, the node resources are cleaned up.

- Main Function

```
def main(args=None):
    rclpy.init(args=args)  # 初始化ROS节点

    minimal_publisher = MinimalPublisher()  # 创建MinimalPublisher对象

    rclpy.spin(minimal_publisher)  # 进入主循环

    minimal_publisher.destroy_node()  # 销毁节点
    rclpy.shutdown()  # 关闭ROS
```

First, invoke the rclpy.init() function to initialize ROS2 Python interface. Then instantiate the MinimalPublisher() file. Finally, execute the minimal_publisher within the event loop of the ROS2 node.

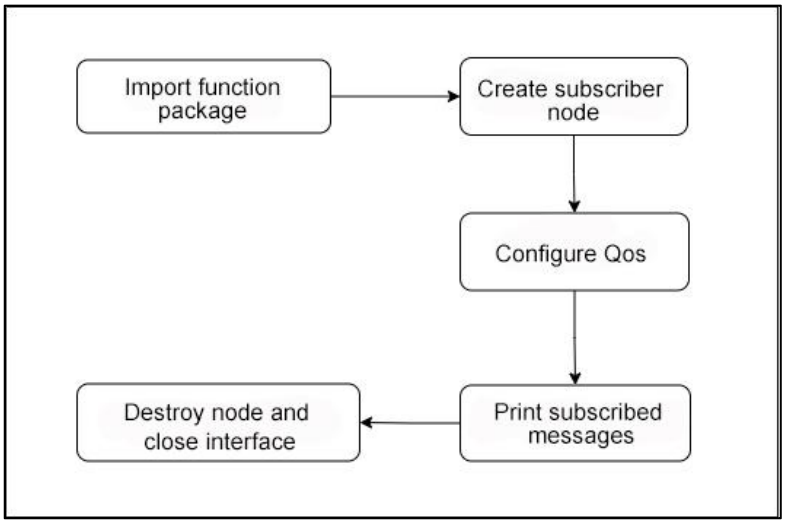
- MinimalPublisher Class

```python
class MinimalPublisher(Node):  # 定义一个继承自Node类的MinimalPublisher类

    def __init__(self):
        super().__init__('minimal_publisher')  # 调用父类构造函数初始化节点
        qos_profile = QoSProfile(  # 创建QoSProfile对象
            reliability=QoSReliabilityPolicy.RELIABLE,  # 设置可靠性策略为RELIABLE
            history=QoSHistoryPolicy.KEEP_LAST,  # 设置历史策略为KEEP_LAST
            depth=1  # 设置深度为1
        )
        self.publisher_ = self.create_publisher(String, 'topic', qos_profile)  # 创建发布者对象
        timer_period = 0.5  # 设置定时器回调的时间间隔为0.5秒
        self.timer = self.create_timer(timer_period, self.timer_callback)  # 创建定时器，设置回调函数为timer_callback
        self.i = 0

    def timer_callback(self):
        msg = String()  # 创建String消息类型的对象
        msg.data = 'Hello World: %d' % self.i  # 设置消息的数据
        self.publisher_.publish(msg)  # 发布消息
        self.i += 1  # 自增计数器
```

Firstly, a MinimalPublisher node class is created to achieve cyclic publishing in ROS 2 reliable mode. In the constructor init(), a QoSProfile object is created to configure the publishing quality to reliable mode. Then, the publisher_ object is initialized using the QoSProfile. The constructor also sets the timer period to 0.5s and creates a timer object. The counter i is initialized to 0. The timer callback function timer_callback is defined, where a String message object msg is first constructed. The current value of the counter i is then written into the message content using string formatting. Finally, this message is published using the publisher_ object defined earlier, and the counter i is incremented by 1.

7.2 Subscribe to Topic

According to the realization result, the logic progress for the program is shown as pictured:



A MinimalSubscriber node class is created to realize the reliable mode subscription in ROS2. In its constructor, it first utilizes a QosProfile object to configure the subscription quality, setting reliability to RELIABLE, retaining the most recent message in history, and setting the buffer depth to 1. Then, creates a subscriber object using the QoSProfile object, subscribes to the topic, and specifies the callback function. The callback function, listener_callback, simply prints the received message content. The main function initializes the node, creates an instance of the subscriber, enters the main loop to drive the callback function, and releases node resources. Through the configuration of QoSProfile, the subscriber ensures that the callback function receives the most recent correct message.

- Main Function

```
def main(args=None):
    rclpy.init(args=args)  # 初始化ROS节点

    minimal_subscriber = MinimalSubscriber()  # 创建MinimalSubscriber对象

    rclpy.spin(minimal_subscriber)  # 进入主循环

    minimal_subscriber.destroy_node()  # 销毁节点
    rclpy.shutdown()  # 关闭ROS
```

First, invoke the rclpy.init() function to initialize ROS2 Python interface. Then instantiate the MinimalSublisher() file. Finally, execute the minimal_sublisher within the event loop of the ROS2 node.

- MinimalSubscriber Class

```
class MinimalSubscriber(Node):  # 定义一个继承自Node类的MinimalSubscriber类

    def __init__(self):
        super().__init__('minimal_subscriber')  # 调用父类构造函数初始化节点
        qos_profile = QoSProfile(  # 创建QoSProfile对象
            reliability=QoSReliabilityPolicy.RELIABLE,  # 设置可靠性策略为RELIABLE
            history=QoSHistoryPolicy.KEEP_LAST,  # 设置历史策略为KEEP_LAST
            depth=1  # 设置深度为1
        )
        self.subscription = self.create_subscription(String, 'topic', self.listener_callback, qos_profile)  # 创建订阅者对象，并设置回调函数为listener_callback

    def listener_callback(self, msg):
        self.get_logger().info('I heard: "%s"' % msg.data)  # 打印接收到的消息内容
```

A MinimalSubscriber node class is first created to implement reliable subscription functionality in ROS2. In its constructor init(), a QoSProfile object is initially created to configure the subscription quality to reliable mode

(RELIABLE, KEEP_LAST, etc.). Then, using the QoSProfile object and the specified callback function, a subscription object is created to subscribe to a topic and specify the callback function. The subscription callback function listener_callback is defined to simply print the received message content msg.